

计算机网络实验报告

课程名称 计算机网络 成绩评定
实验项目名称 Wireshark 综合实验 (2) 指导教师 张伟
实验项目编号 实验 5 实验项目类型 实验地点 淦昌苑 D 座 226
学生姓名 李文峰 学号 202300460158
学院 网络空间安全 专业 网络空间安全
实验时间 2025 年 4 月 23 日

一、实验目的

1. 熟悉 TCP 的协议格式。
2. 理解 TCP 对序列号和确认号的使用。
3. 理解 TCP 的流量控制算法和拥塞控制算法。

二、实验步骤与结果

任务一：捕获从计算机到远程服务器的批量 TCP 传输

打开浏览器下载相应文档，将该文档上传到指定网页中，启动 wireshark 开始数据包捕获，等待发送完成后停止捕获。输入 tcp 过滤数据包以查看 TCP 和 HTTP 通信。

1.此时可以看到 TCP 三次握手建立连接的过程，如下图所示。在建立连接中，第一次握手客户端向服务端发送请求报文 SYN=1、Seq=0；第二次握手时，服务端发送确认报文 ACK=1、SYN=1、Seq=0；第三次握手时，客户端再次发送确认报文，ACK=1、Seq=1.

No.	Time	Source	Destination	Protocol	Length	Info
21	0.277138	172.25.227.113	111.30.170.28	TCP	66	52102 → 8081 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=2 SACK_PERM
22	0.293769	20.114.190.119	172.25.227.113	TLSv1.2	85	Encrypted Alert
23	0.293934	20.114.190.119	172.25.227.113	TCP	56	443 → 52097 [FIN, ACK] Seq=32 Ack=1 Win=501 Len=0
24	0.293993	172.25.227.113	20.114.190.119	TCP	54	52097 → 443 [ACK] Seq=1 Ack=33 Win=254 Len=0
25	0.300037	111.30.170.28	172.25.227.113	TCP	66	8081 → 52102 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1424 SACK_PERM WS=512
26	0.300148	172.25.227.113	111.30.170.28	TCP	54	52102 → 8081 [ACK] Seq=1 Ack=1 Win=65536 Len=0
27	0.300228	172.25.227.113	111.30.170.28	TCP	218	52102 → 8081 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=164
31	0.339155	111.30.170.28	172.25.227.113	TCP	56	8081 → 52102 [ACK] Seq=1 Ack=165 Win=67072 Len=0
32	0.339155	111.30.170.28	172.25.227.113	TCP	80	8081 → 52102 [PSH, ACK] Seq=1 Ack=165 Win=67072 Len=26
34	0.386776	172.25.227.113	111.30.170.28	TCP	54	52102 → 8081 [ACK] Seq=165 Ack=27 Win=65510 Len=0
35	0.402909	172.25.227.113	120.233.20.242	TCP	54	52101 → 36688 [FIN, ACK] Seq=1 Ack=1 Win=32748 Len=0
45	0.515487	120.233.20.242	172.25.227.113	TCP	56	36688 → 52101 [FIN, ACK] Seq=1 Ack=2 Win=133 Len=0
46	0.515487	20.114.190.119	172.25.227.113	TLSv1.2	85	Encrypted Alert
47	0.515487	20.114.190.119	172.25.227.113	TCP	56	443 → 52096 [FIN, ACK] Seq=32 Ack=1 Win=501 Len=0
48	0.515629	172.25.227.113	120.233.20.242	TCP	54	52101 → 36688 [ACK] Seq=2 Ack=2 Win=32748 Len=0
49	0.515750	172.25.227.113	20.114.190.119	TCP	54	52096 → 443 [ACK] Seq=1 Ack=33 Win=253 Len=0
120	1.565949	172.25.227.113	128.119.245.12	TCP	66	52103 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
135	1.822433	172.25.227.113	128.119.245.12	TCP	66	52105 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
157	2.153962	128.119.245.12	172.25.227.113	TCP	66	80 → 52105 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM WS=128
158	2.154116	172.25.227.113	128.119.245.12	TCP	54	52105 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
159	2.154498	172.25.227.113	128.119.245.12	TCP	1514	52105 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 347]
160	2.154498	172.25.227.113	128.119.245.12	TCP	1514	52105 → 80 [ACK] Seq=1461 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 347]
161	2.154498	172.25.227.113	128.119.245.12	TCP	1514	52105 → 80 [ACK] Seq=2921 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 347]
162	2.154498	172.25.227.113	128.119.245.12	TCP	1514	52105 → 80 [ACK] Seq=4381 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 347]
163	2.154498	172.25.227.113	128.119.245.12	TCP	1514	52105 → 80 [ACK] Seq=5841 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 347]
164	2.154498	172.25.227.113	128.119.245.12	TCP	1514	52105 → 80 [ACK] Seq=7301 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 347]

2.在 TCP 连接结束时，包含 FIN 的 TCP 连接关闭请求如下图所示。

No.	Time	Source	Destination	Protocol	Length	Info
450	5.852942	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=131401 Win=250368 Len=0
451	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=132861 Win=253312 Len=0
452	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=134321 Win=256256 Len=0
453	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=135781 Win=259200 Len=0
454	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=137241 Win=262144 Len=0
455	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=138701 Win=264960 Len=0
456	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=140161 Win=267904 Len=0
457	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=141621 Win=270848 Len=0
458	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=143081 Win=273792 Len=0
459	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=146001 Win=279552 Len=0
460	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=147461 Win=282496 Len=0
461	5.853114	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=148921 Win=285440 Len=0
462	5.854878	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=150381 Win=288384 Len=0
463	5.854878	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=151841 Win=291328 Len=0
464	5.854934	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=1 Ack=153008 Win=292096 Len=0
465	5.854934	128.119.245.12	172.25.227.113	HTTP	831	HTTP/1.1 200 OK (text/html)
467	5.902408	172.25.227.113	128.119.245.12	TCP	54	53812 → 80 [ACK] Seq=153008 Ack=778 Win=64512 Len=0
723	10.525206	172.25.227.113	128.119.245.12	TCP	54	53813 → 80 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
732	10.835582	128.119.245.12	172.25.227.113	TCP	56	80 → 53813 [FIN, ACK] Seq=1 Ack=2 Win=29312 Len=0
733	10.835657	172.25.227.113	128.119.245.12	TCP	54	53813 → 80 [ACK] Seq=2 Ack=2 Win=65280 Len=0
734	10.843697	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [FIN, ACK] Seq=778 Ack=153008 Win=292096 Len=0
735	10.843767	172.25.227.113	128.119.245.12	TCP	54	53812 → 80 [ACK] Seq=153008 Ack=779 Win=64512 Len=0
736	10.843891	172.25.227.113	128.119.245.12	TCP	54	53812 → 80 [FIN, ACK] Seq=153008 Ack=779 Win=64512 Len=0
758	11.223858	172.25.227.113	128.119.245.12	TCP	54	[TCP Retransmission] 53812 → 80 [FIN, ACK] Seq=153008 Ack=779 Win=
759	11.232079	128.119.245.12	172.25.227.113	TCP	56	80 → 53812 [ACK] Seq=779 Ack=153009 Win=292096 Len=0
778	11.585110	128.119.245.12	172.25.227.113	TCP	56	[TCP Dup ACK 759#1] 80 → 53812 [ACK] Seq=779 Ack=153009 Win=292096

任务二：打开 Wireshark 抓包文件 tcp-ethereal-trace-1 回答以下问题

1.将文件传输到客户端计算机（源）使用的 IP 地址和 TCP 端口号是什么？

经过观察，源 IP 地址为 192.168.1.102；其 TCP 端口号为 1161。如图所示。

```

> Frame 1: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
> Ethernet II, Src: ActiontecEle_8a:70:1a (00:20:e0:8a:70:1a), Dst: LinksysGroup_da:af:73 (0
> Internet Protocol Version 4, Src: 192.168.1.102, Dst: 128.119.245.12
v Transmission Control Protocol, Src Port: 1161, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 1161
  Destination Port: 80
  [Stream index: 0]
  [Stream Packet Number: 1]
  > [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 232129012
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0

```

2.gaia.cs.umass.edu 的 IP 地址是什么？在哪个端口号上发送和接收此连接的 TCP 报文段？

客户端 gaia.cs.umass.edu 的 IP 地址为 128.119.245.12；端口号为 80.

```

> Frame 1: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
> Ethernet II, Src: ActiontecEle_8a:70:1a (00:20:e0:8a:70:1a), Dst: LinksysGroup_da:af:73 (00:06:25:da:af:73)
> Internet Protocol Version 4, Src: 192.168.1.102, Dst: 128.119.245.12
v Transmission Control Protocol, Src Port: 1161, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 1161
  Destination Port: 80
  [Stream index: 0]
  [Stream Packet Number: 1]
  > [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 232129012
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0

```

任务三：继续使用 Wireshark 抓包文件 tcp-ethereal-trace-1 执行分析

1.用于在客户端计算机和 gaia.cs.umass.edu 之间启动 TCP 连接的 TCP SYN 报文段的序列号是什么？TCP SYN 报文段有什么作用？

启动 TCP 连接的 TCP SYN 报文段的序列号为 **232129012**。作用：客户端通过发送 SYN 报文主动向服务器发起连接请求，表示希望建立 TCP 连接。SYN 报文中还包含客户端的初始序列号，用于后续数据传输的顺序控制，同时 SYN 报文是 TCP 三次握手的第一个步骤，确保双方通信能力正常且参数一致。

Protocol	Length	Info
TCP	62	1161 → 80 [SYN] Seq=232129012 Win=16384 Len=0 MSS
TCP	62	80 → 1161 [SYN, ACK] Seq=883061785 Ack=232129013
TCP	54	1161 → 80 [ACK] Seq=232129013 Ack=883061786 Win=1
TCP	619	1161 → 80 [PSH, ACK] Seq=232129013 Ack=883061786
TCP	1514	1161 → 80 [PSH, ACK] Seq=232129578 Ack=883061786
TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232129578 Win=6
TCP	1514	1161 → 80 [ACK] Seq=232131038 Ack=883061786 Win=1
TCP	1514	1161 → 80 [ACK] Seq=232132498 Ack=883061786 Win=1
TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232131038 Win=8
TCP	1514	1161 → 80 [ACK] Seq=232133958 Ack=883061786 Win=1

2.gaia.cs.umass.edu 发送给客户端计算机以回复 SYN 的 SYN-ACK 报文段的序列号是多少？SYN ACK 报文段中的 Acknowledgment number 栏位的值是多少？gaia.cs.umass.edu 是如何确定此 Acknowledgment number 的数值的？TCP SYN-ACK 报文段有什么作用？

序列号为 **883061785**，Acknowledgment number 的值为 **232129013**。服务端在接收到 SYN 报文连接请求后，查看第一次握手序列号 Seq，该序列号的值为 232129012，所以服务端 gaia.cs.umass.edu 将该序列号加 1，得到的值作为第二次握手的 Acknowledgment number 值（即 ACK 号）。SYN-ACK 报文段用于确认客户端的 SYN 请求，同时同步服务端的初始序列号。

```
> [Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 0]
Sequence Number: 883061785
[Next Sequence Number: 883061786]
Acknowledgment Number: 232129013
0111 .... = Header Length: 28 bytes (7)
> Flags: 0x012 (SYN, ACK)
Window: 5840
[Calculated window size: 5840]
Checksum: 0x774d [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> Options: (8 bytes), Maximum segment size, No-Operation (NOP), No-
> [Timestamps]
> [SEQ/ACK analysis]
```

3.包含 HTTP POST 命令的 TCP 报文段的序列号是多少？

通过观察，第 4 个报文即为 HTTP POST 命令的 TCP 报文，序列号为 232129013.

```
> [Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 565]
Sequence Number: 232129013
[Next Sequence Number: 232129578]
Acknowledgment Number: 883061786
0101 .... = Header Length: 20 bytes (5)
> Flags: 0x018 (PSH, ACK)
Window: 17520
[Calculated window size: 17520]
[Window size scaling factor: -2 (no window scaling used)]
Checksum: 0x1fbd [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> [Timestamps]
> [SEQ/ACK analysis]
TCP payload (565 bytes)
▼ Data (565 bytes)
Data [...]: 504f5354202f6574686572655616c2d6c6162732f6c6162332d312d7265706c792e68746d;
[Length: 565]
```

0000	00 06 25 da af 73 00 20	e0 8a 70 1a 08 00 45 00	..%..s. .p...E.
0010	02 5d 1e 21 40 00 80 06	a2 e7 c0 a8 01 66 80 77	..]!@... ..f.w
0020	f5 0c 04 89 00 50 0d d6	01 f5 34 a2 74 1a 50 18	P... ..4.t.P.
0030	44 70 1f bd 00 00 50 4f	53 54 20 2f 65 74 68 65	Dp...PO ST ethe
0040	72 65 61 6c 2d 6c 61 62	73 2f 6c 61 62 33 2d 31	real-lab s/lab3-1
0050	2d 72 65 70 6c 79 2e 68	74 6d 20 48 54 54 50 2f	-reply.h tm HTTP/
0060	31 2e 31 0d 0a 48 6f 73	74 3a 20 67 61 69 61 2e	1.1..Hos t: gaia.
0070	63 73 2e 75 6d 61 73 73	2e 65 64 75 0d 0a 55 73	cs.umass .edu..Us
0080	65 72 2d 41 67 65 6e 74	3a 20 4d 6f 7a 69 6c 6c	er-Agent : Mozill
0090	61 2f 35 2e 30 20 28 57	69 6e 64 6f 77 73 3b 20	a/5.0 (W indows;
00a0	55 3b 20 57 69 6e 64 6f	77 73 20 4e 54 20 35 2e	U; Windo ws NT 5.
00b0	31 3b 20 65 6e 2d 55 53	3b 20 72 76 3a 31 2e 30	1; en-US ; rv:1.0
00c0	2e 32 29 20 47 65 63 6b	6f 2f 32 30 30 33 30 32	.2) Geck o/200302
00d0	30 38 20 4e 65 74 73 63	61 70 65 2f 37 2e 30 32	08 Netsc ape/7.02
00e0	0d 0a 41 63 63 65 70 74	3a 20 74 65 78 74 2f 78	..Accept : text/x
00f0	6d 6c 2c 61 70 70 6c 69	63 61 74 69 6f 6e 2f 78	ml,appli cation/x
0100	6d 6c 2c 61 70 70 6c 69	63 61 74 69 6f 6e 2f 78	ml appli cation/x

4.观察编号 3 和 4 的 TCP 报文段的序列号，你有什么发现？请解释原因。

编号为 3 和 4 的 TCP 报文段的序列号 Seq、确认号 Ack、Win 等值均相同，但编号 3 的长度 Len 为 0，编号 4 的 Len 值为 565. 此外第 3 个报文为[ACK], 第 4 个报文为[PSH, ACK].

原因：编号为 3 的报文段是第三次握手确认，其不携带数据仅确认连接所以 Len 等于 0. 而编号为 4 的报文段是握手完成后发送数据，其中 Len=565 代表着数据长度。由于编号为 3 的报文段是纯确认报文，不占用序列号，故编号为 4 的报

文起始序列号与前一报文相同。由于第三行到第四行时，服务端还没有发送任何新数据，所以 ACK 保持不变。

! Lengua info

62	1161	→	80	[SYN]	Seq=232129012	Win=16384	Len=0	MSS=1460	SACK_PERM	
62	80	→	1161	[SYN, ACK]	Seq=883061785	Ack=232129013	Win=5840	Len=0	MSS=1460	SACK_PERM
54	1161	→	80	[ACK]	Seq=232129013	Ack=883061786	Win=17520	Len=0		
619	1161	→	80	[PSH, ACK]	Seq=232129013	Ack=883061786	Win=17520	Len=565		
1514	1161	→	80	[PSH, ACK]	Seq=232129578	Ack=883061786	Win=17520	Len=1460		
60	80	→	1161	[ACK]	Seq=883061786	Ack=232129578	Win=6780	Len=0		
1514	1161	→	80	[ACK]	Seq=232131038	Ack=883061786	Win=17520	Len=1460		
1514	1161	→	80	[ACK]	Seq=232132498	Ack=883061786	Win=17520	Len=1460		
60	80	→	1161	[ACK]	Seq=883061786	Ack=232131038	Win=8760	Len=0		
1514	1161	→	80	[ACK]	Seq=232133958	Ack=883061786	Win=17520	Len=1460		

5.将包含 HTTP POST 的 TCP 报文段视为 TCP 连接中的第一个报文段。在这个 TCP 连接中前六个用于数据发送的 TCP 报文段是那些？序列号分别是多少？这六个报文段发送的时间是什么时候？

前六个用于数据发送的 TCP 报文段分别为编号为 4、5、7、8、10、11. 序列号分别为 232129013、232129578、232131038、232132498、232133958、232135418. 这六个报文段发送的时间分别是：第 0.026477s、0.041737s、0.054026s、0.054690s、0.077405s、0.078157s.

6. 收到的对应前六个数据发送 TCP 报文段的确认 ACK 分别是在那些 TCP 报文段里（列出编号，即查看“No.”这一列）？是什么时候收到的？

用于确认前六个用于数据发送的 TCP 报文为如下：

- No.6, 到达时间为 Aug 21, 2004 21:44:20.624318000 中国标准时间
- No.9, 到达时间为 Aug 21, 2004 21:44:20.647675000 中国标准时间
- No.12, 到达时间为 Aug 21, 2004 21:44:20.694466000 中国标准时间
- No.14, 到达时间 Aug 21, 2004 21:44:20.739499000 中国标准时间

- No.15, 到达时间 Aug 21, 2004 21:44:20.787680000 中国标准时间
- No.16, 到达时间 Aug 21, 2004 21:44:20.838183000 中国标准时间

7.鉴于发送每个 TCP 报文段的时间与收到确认的时间之间的差异，前六个数据发送 TCP 报文段中每个报文段的往返时间（RTT）是多少？加权平均往返时延（RTTS）是多少？

各报文段 RTT:

No.6 确认 No.4 报文: $0.053937 - 0.026477 = 0.027460s$

No.9 确认 No.5 报文: $0.077294 - 0.041737 = 0.035557s$

No.12 确认 No.7 报文: $0.124085 - 0.054026 = 0.070059s$

No.14 确认 No.8 报文: $0.169118 - 0.054690 = 0.114428s$

No.15 确认 No.10 报文: $0.217299 - 0.077405 = 0.139894s$

No.16 确认 No.11 报文: $0.267802 - 0.078157 = 0.189645s$

加权平均往返时延 RTTS 为: $0.072574s$

8.前六个数据发送 TCP 报文段的长度是多少？

前六个数据发送 TCP 报文段的长度为: 565、1460、1460、1460、1460、1460.

9.对于整个抓包过程，收到的服务器声明的最小可用接收缓冲区空间（接收窗口）大小是多少？整个过程中，声明的接收缓冲区空间是否限制了发送方传送

TCP 报文段？

最小可用接受缓冲区空间（接收窗口）大小为 6780，在第 6 个报文中出现最小接收窗口。从整个抓包过程来看，服务器声明的接收窗口大小并没有限制发送方传送 TCP 报文段，因为接收窗口在传输中有所增大，因此未限制发送。

10.在抓包文件中是否有重传的报文段？为了回答这个问题，你检查了什么？

抓包文件中没有重传的报文段，所有数据报文段的序列号 Seq 连续递增，且 ACK 号严格对应，且对同一报文的确认报文没有重复发送多次，

11.接收方通常在 ACK 中确认多少数据（确认收到的多少个报文段）？你是否可以识别接收方每隔一个接收到的报文段才发送确认 ACK 的情况？

接收方通常确认 1460 的数据量；通过实验可以看出，接收方确实是每隔一个接收到的报文段才发送确认，例如下图所示，对于第 7 个报文的确认直到在第 14 个报文时才发送，其间又隔着新的发送报文。

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.102	128.119.245.12	TCP	62	1161 → 80 [SYN] Seq=232129012 Win=16384 Len=0 MSS=1460 SACK_PERM
2	0.023172	128.119.245.12	192.168.1.102	TCP	62	80 → 1161 [SYN, ACK] Seq=883061785 Ack=232129013 Win=5840 Len=0 MSS=1460 SACK_PERM
3	0.023265	192.168.1.102	128.119.245.12	TCP	54	1161 → 80 [ACK] Seq=232129013 Ack=883061786 Win=17520 Len=0
4	0.026477	192.168.1.102	128.119.245.12	TCP	619	1161 → 80 [PSH, ACK] Seq=232129013 Ack=883061786 Win=17520 Len=565
5	0.041737	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [PSH, ACK] Seq=232129578 Ack=883061786 Win=17520 Len=1460
6	0.053937	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232129578 Win=6780 Len=0
7	0.054026	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232131038 Ack=883061786 Win=17520 Len=1460
8	0.054690	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232132498 Ack=883061786 Win=17520 Len=1460
9	0.077294	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232131038 Win=8760 Len=0
10	0.077405	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232133958 Ack=883061786 Win=17520 Len=1460
11	0.078157	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232135418 Ack=883061786 Win=17520 Len=1460
12	0.124085	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232132498 Win=11680 Len=0
13	0.124185	192.168.1.102	128.119.245.12	TCP	1201	1161 → 80 [PSH, ACK] Seq=232136878 Ack=883061786 Win=17520 Len=1147
14	0.169118	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232133958 Win=14600 Len=0
15	0.217299	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232135418 Win=17520 Len=0
16	0.267802	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232136878 Win=20440 Len=0
17	0.304807	128.119.245.12	192.168.1.102	TCP	60	80 → 1161 [ACK] Seq=883061786 Ack=232138025 Win=23360 Len=0
18	0.305040	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232138025 Ack=883061786 Win=17520 Len=1460
19	0.305813	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232139485 Ack=883061786 Win=17520 Len=1460
20	0.306692	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232140945 Ack=883061786 Win=17520 Len=1460
21	0.307571	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232142405 Ack=883061786 Win=17520 Len=1460
22	0.308699	192.168.1.102	128.119.245.12	TCP	1514	1161 → 80 [ACK] Seq=232143865 Ack=883061786 Win=17520 Len=1460
23	0.309553	192.168.1.102	128.119.245.12	TCP	946	1161 → 80 [PSH, ACK] Seq=232145325 Ack=883061786 Win=17520 Len=892

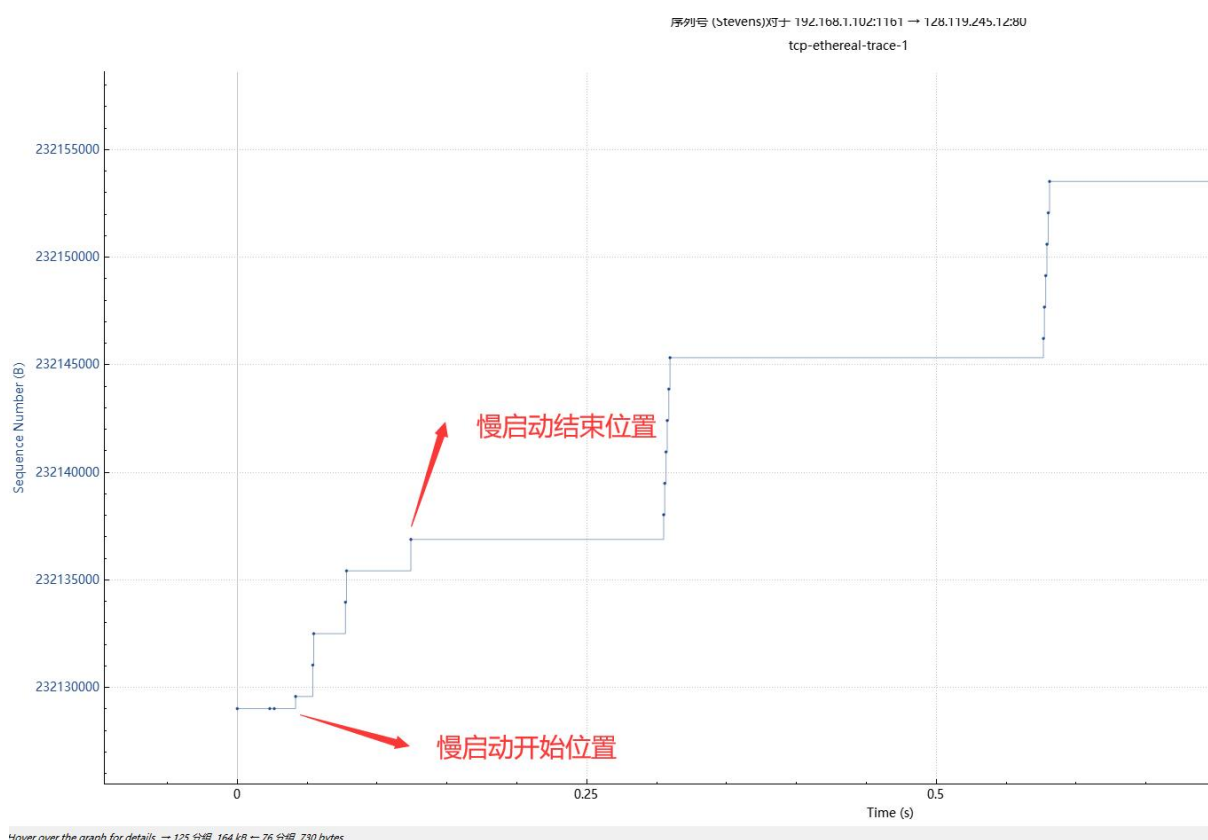
12.TCP 连接的吞吐量（每单位时间传输的字节数）是多少？解释你如何计算这个值。

吞吐量=数据量/传输时间=164090 字节/5.4294s=30.222KB/s

任务四：分析 tcp-ethereal-trace-1 的数据量

1.使用时间序列图绘图工具查看从客户端发送到服务器的报文段的序列号与时间关系图。能否确定 TCP 的慢启动阶段的开始和结束位置，以及拥塞避免接管的位置？

图像刚开始为慢启动阶段，具体地开始和结束位置如图所示。之后开始较为平稳有规律地的增长，其中包含拥塞控制。



2.评论测量数据与我们在书本中研究的 TCP 的理想化行为的不同之处
不同之处为慢启动阶段并非严格指数增长且未发生拥塞